Математички методи за машинско учење 2025/26

Домаћи задатак број 8

Име и презиме студента: Jovan Paunović

Број индекса: 2122

Упутство за израду и предају домаћег задатака:

1. Пре почетка израде промените име датотеке у 08Domaci_Ime_Prezime. (убаците своје име и презиме)
2. Попуните ћелију испод наслова одговарајућим подацима.
1. Употреба ћирилице није обавезна за предају домаћег задатка.
4. За решавање задатака, уколико је потребно, отворите испод текста задатка додатне ћелије за уписивање текстуалног одговора (Markdown) или програмског кода (Code).
1. Сва израчунавања, уколико је потребно, вршити у Python-у.
5. Након завршетка израде решења домаћег Notebook документ сачувати у pdf формату и проследити га наставнику. То можете да урадите или кроз Teams или на мејл адресу jovana.dzunic@elfak.ni.ac.rs

In [2]:
```
import numpy as np
import numpy.random as rndm
import matplotlib as mplb
import matplotlib.pyplot as plt
from timeit import default_timer as timer
```

Задатак 1. Нека је $A \in \mathcal{M}_{I\times J\times K}$ тензор трећег реда. Фробенијусова норма тензора дефинише се као корен суме квадрата свих његових елемената:

$$\|A\|_F = \sqrt{\sum_{i,j,k} a_{i,j,k}^2}.$$

а) Доказати да је Фробенијусова норма тензора идентична Фробенијусовој норми било ког његовог расклапања по моду, тј.

$$\|A\|_F = \|A_{[0]}\|_F = \|A_{[1]}\|_F = \|A_{[2]}\|_F.$$

(5 поена)

Frobenijusova norma matrice M se definiše kao kvadratni koren sume kvadrata svih njenih elemenata:

$$\|M\|_F = \sqrt{\sum_{p,q} m_{p,q}^2}$$

Rasklapanje tenzora A po nekom modu n, koje označavamo sa $A_{[n]}$, predstavlja operaciju reorganizacije elemenata tog tenzora u dvodimenzionalnu matricu.

Prilikom ove operacije menja se isključivo dimenzionalnost i raspored elemenata, dok same vrednosti elemenata i njihov ukupan broj ostaju nepromenjeni. Zbog te činjenice, suma kvadrata svih elemenata matrice $A_{[n]}$ jednaka je sumi kvadrata svih elemenata originalnog tenzora A:

$$\sum_{p,q} (A_{[n]})_{p,q}^2 = \sum_{i,j,k} a_{i,j,k}^2$$

Ako na ovu jednakost primenimo kvadratni koren na obe strane, dobijamo:

$$\sqrt{\sum_{p,q} (A_{[n]})_{p,q}^2} = \sqrt{\sum_{i,j,k} a_{i,j,k}^2}$$

Leva strana jednakosti po definiciji predstavlja Frobenijusovu normu matrice $\|A_{[n]}\|_F$, dok desna strana po definiciji predstavlja Frobenijusovu normu tenzora $\|A\|_F$.

Odatle direktno sledi da za svaki mod $n \in \{0, 1, 2\}$ važi:

$$\|A_{[n]}\|_F = \|A\|_F$$

5/5поена

б) Направити нумерички експеримент: генерисати случајан тензор димензија $50 \times 40 \times 30$, израчунати норму директно и преко сва три матрична расклапања користећи `np.reshape` и `np.transpose`. Упоредити резултате.

(5 поена)

In [6]:
```
I, J, K = 50, 40, 30
A = np.random.rand(I, J, K)

tensor_norm = np.linalg.norm(A)

A_0 = A.reshape(I, J * K)
norm_0 = np.linalg.norm(A_0)

A_1 = np.transpose(A, (1, 0, 2)).reshape(J, I * K)
norm_1 = np.linalg.norm(A_1)

A_2 = np.transpose(A, (2, 0, 1)).reshape(K, I * J)
norm_2 = np.linalg.norm(A_2)

print(f"Norma tenzora: {tensor_norm:.10f}")
print(f"Norma A_[0]: {norm_0:.10f}")
print(f"Norma A_[1]: {norm_1:.10f}")
print(f"Norma A_[2]: {norm_2:.10f}")
```

```
Norma tenzora: 141.2060984647
Norma A_[0]: 141.2060984647
Norma A_[1]: 141.2060984647
Norma A_[2]: 141.2060984647
```

Kao što je i očekivano, rezultati su isti.

5/5поена

Задатак 2. Дати су тензори $A \in \mathcal{M}_{B\times N\times D}$ и $B \in \mathcal{M}_{B\times D\times M}$. Овакав облик често срећемо у "Attention" механизмима где је B batch size.

а) Израчунати тензор
$C \in \mathcal{M}_{B\times N\times M}$
који представља матрично множење A и B за сваки елемент batch-а посебно (Batch Matrix Multiplication), користећи:

- `np.einsum`
- `@`
- `np.tensordot` (уз неопходне додатне операције ако су потребне)

(5 поена)

In [11]:

```
B, N, D, M = 10, 5, 4, 6

A = np.random.rand(B, N, D)
B = np.random.rand(B, D, M)

C_einsum = np.einsum('bnd,bdm->bnm', A, B)
print(C_einsum.shape)

C_matmul = A @ B
print(C_matmul.shape)

C_td_raw = np.tensordot(A, B, axes=([2], [1])) # dobija se tenzor dimenz
print(C_td_raw.shape)

# sada moramo da izvučemo "dijagonalu" preko dve batch dimenzije (nulta
# jer nas zanimaju samo oni slučajevi gde je batch indeks iz A jednak ba
# nakon primene np.diagonal dobijamo oblik (N, M, B).
C_td_diag = np.diagonal(C_td_raw, axis1=0, axis2=2)

# vršimo transponovanje da vratimo batch dimenziju na prvo mesto: (B, N,
C_tensordot = np.transpose(C_td_diag, (2, 0, 1))
print(C_tensordot.shape)

print("C_einsum == C_matmul?", np.allclose(C_einsum, C_matmul))
print("C_tensordot == C_matmul?", np.allclose(C_tensordot, C_matmul))
print(f"Dimenzije rezultata: {C_matmul.shape}")
```

```
(10, 5, 6)
(10, 5, 6)
(10, 5, 10, 6)
(10, 5, 6)
C_einsum == C_matmul? True
C_tensordot == C_matmul? True
Dimenzije rezultata: (10, 5, 6)
```

5/5поена

б) Измерити време извршавања за димензије
$B = 100, N = 128, D = 256, M = 128$
за сва три приступа. Који је приступ најчитљивији, а који најбржи?

(5 поена)

In [12]:
```
B, N, D, M = 100, 128, 256, 128

A = np.random.rand(B, N, D)
B = np.random.rand(B, D, M)

einsum_start = timer()
C_einsum = np.einsum('bnd,bdm->bnm', A, B)
einsum_time = timer() - einsum_start

matmul_start = timer()
C_matmul = A @ B
matmul_time = timer() - matmul_start

tensor_start = timer()
C_td_raw = np.tensordot(A, B, axes=([2], [1]))
C_td_diag = np.diagonal(C_td_raw, axis1=0, axis2=2)
C_tensordot = np.transpose(C_td_diag, (2, 0, 1))
tensor_time = timer() - tensor_start

print("Vreme einsum: ", einsum_time)
print("Vreme @: ", matmul_time)
print("Vreme tensordot:", tensor_time)
```

```
Vreme einsum:  0.2889207999687642
Vreme @:  0.022639800095930696
Vreme tensordot: 1.4815926000010222
```

Najbrži pristup je korišćenje `@` . Nakon toga dolazi korišćenje `np.einsum` , dok je na poslednjem mestu korišćenje `np.tensordot` . Iako `@` zahteva najkraći kod, može da deluje zbunjujuće koji će biti efekat te naredbe. `np.tensordot` nije čitljiv i zahteva dodatne korake. Najčitljivija opcija je korišćenje `np.einsum` jer jasno vidimo koje su dimenzije ulaznih tenzora i koje dimenzije ćemo dobiti kao rezultat.

5/5поена

Задатак 3. У материјалима је дефинисан производ тензора и матрице по моду k као

$$(A \times_k M)_{[k]} = M A_{[k]}.$$

а) Ако је

$$X = G \times_1 A \times_2 B \times_3 C$$

(што је основа Tucker декомпозиције), показати да се расклапање по првом моду може записати као:

$$X_{[0]} = A\, G_{[0]} (C \otimes B)^T.$$

Сугестија: користити особину $(A \times_k M)^{[k]} = M A^{[k]}$ и везу Кронекеровог производа са векторизацијом.

(10 поена)

Krećemo od definicije tenzora X:

$$X = G \times_1 A \times_2 B \times_3 C$$

Uvedimo pomoćni tenzor Y koji predstavlja množenje tenzora G matricama B i C po drugom i trećem modu (odnosno $k = 2$ i $k = 3$):

$$Y = G \times_2 B \times_3 C$$

Na osnovu rada (https://www.kolda.net/publication/TensorReview.pdf) - sekcija 2.5, važi $\mathcal{X} \times_n A \times_m B = \mathcal{X} \times_m B \times_n A \quad$ gde je $\quad n \neq m$. Dakle, početni tenzor se može zapisati jednostavnije kao:

$$X = Y \times_1 A$$

Na osnovu date osobine za proizvod tenzora i matrice po istom modu po kom se vrši rasklapanje ($(A \times_k M)_{[k]} = M A_{[k]}$), za mod 1 (indeksiran sa 0 u oznaci rasklapanja $X_{[0]}$) važi:

$$X_{[0]} = (Y \times_1 A)_{[0]} = A Y_{[0]}$$

Sada je cilj da izrazimo matricu $Y_{[0]}$ preko $G_{[0]}$, B i C. Posmatrajmo elemente tenzora Y. Za fiksiran indeks prvog moda i, elementi $y_{i,j,k}$ formiraju dvodimenzionalnu matricu (presek) koju možemo označiti sa Y_i. Na osnovu definicije množenja po modovima 2 i 3, ova matrica se dobija množenjem odgovarajuće matrice G_i (iz tenzora G) matricama B (sleva) i C (zdesna transponovano):

$$Y_i = B G_i C^T$$

Rasklapanje tenzora po prvom modu (indeks 0, $Y_{[0]}$) rezultuje matricom u kojoj svaka i-ta vrsta zapravo predstavlja vektorizovanu matricu preseka Y_i. Dakle, i-ti red matrice $Y_{[0]}$ je transponovani vektor $\mathrm{vec}(Y_i)^T$.

Ovde koristimo poznatu vezu između Kronekerovog proizvoda i vektorizacije matrica, koja u opštem slučaju glasi $\mathrm{vec}(MNP) = (P^T \otimes M)\mathrm{vec}(N)$.

Primenom ovog identiteta na našu matricu Y_i, imamo:

$$\mathrm{vec}(Y_i) = \mathrm{vec}(B G_i C^T) = ((C^T)^T \otimes B)\mathrm{vec}(G_i) = (C \otimes B)\mathrm{vec}(G_i)$$

Transponovanjem obe strane jednakosti dobijamo izraz za vrste rasklopljene matrice:

$$\mathrm{vec}(Y_i)^T = \mathrm{vec}(G_i)^T (C \otimes B)^T$$

S obzirom na to da ovo važi za svaku vrstu i, a vrste matrice $G_{[0]}$ su zapravo vektori $\mathrm{vec}(G_i)^T$, cele matrice možemo povezati identičnom relacijom:

$$Y_{[0]} = G_{[0]} (C \otimes B)^T$$

Konačno, vraćanjem ovog rezultata u početnu jednačinu za $X_{[0]}$, direktno dobijamo traženi izraz:

$$X_{[0]} = A\left(G_{[0]} (C \otimes B)^T\right) = A G_{[0]} (C \otimes B)^T$$

10/10поена

б) Написати функцију `mode_n_product(T, M, n)` која извршава множење тензора T матрицом M по моду n користећи искључиво `np.einsum` . Тестирати функцију на случајним подацима.

(5 поена)

In [15]:
```
def mode_n_product(T, M, n):
    ndim = T.ndim

    t_chars = [chr(97 + i) for i in range(ndim)] # a, b, c ...
    t_str = "".join(t_chars)

    # slovo za novu dimenziju koju unosi matrica M
    # koristimo 'z' pod pretpostavkom da tenzor nema više od 25 dimenzij
    new_dim_char = 'z'

    # prva dimenzija matrice M je z,
    # a druga dimenzija je ona po kojoj se vrši sakupljanje (n-ta dimenz
    m_str = new_dim_char + t_chars[n]

    # oznake rezultujućeg tenzora (kopiramo indekse od T i menjamo n-ti
    res_chars = t_chars.copy()
    res_chars[n] = new_dim_char
    res_str = "".join(res_chars)

    einsum_string = f"{t_str},{m_str}->{res_str}"

    return np.einsum(einsum_string, T, M)


T = np.random.rand(3, 4, 5)

# želimo da množimo po modu 1
# tenzor na osi 1 ima dimenziju 4, pa matrica M mora imati dimenzije z x
M = np.random.rand(6, 4)

result = mode_n_product(T, M, n=1)

print(f"Dimenzije tenzora T: {T.shape}")
print(f"Dimenzije matrice M: {M.shape}")
print(f"Dimenzije rezultata: {result.shape}")
print(f"Očekivana nova dimenzija (3, 6, 5). Dobijena: {result.shape}")
```

```
Dimenzije tenzora T: (3, 4, 5)
Dimenzije matrice M: (6, 4)
Dimenzije rezultata: (3, 6, 5)
Očekivana nova dimenzija (3, 6, 5). Dobijena: (3, 6, 5)
```

5/5поена

Укупно 35/35поена